

task5 notes

April 26, 2026

```
[1]: import numpy as np
```

0.1 Symmetric Matrices and Numeric Stability

```
[2]: def spectral_decomposition(a):  
    # Compute eigenvalues and eigenvectors  
    eigen_values, eigen_vectors = np.linalg.eig(a)  
  
    # Construct diagonal matrix of eigenvalues  
    lambda_mat = np.diag(eigen_values)  
  
    # Reconstruct matrix: A = Q * Lambda_mat * Q-1  
    q = eigen_vectors  
    q_inv = np.linalg.inv(q)  
    mat_reconstructed = q @ lambda_mat @ q_inv  
  
    return eigen_values, eigen_vectors, mat_reconstructed  
  
A_symmetric = np.array([[6, 2, 1],  
                        [2, 3, 1],  
                        [1, 1, 2]])  
  
print("\n" + "=" * 60)  
print("SYMMETRIC MATRICES")  
print("=" * 60)  
print(f"Symmetric Matrix A:\n{A_symmetric}\n")  
  
    # Verify symmetry  
    is_symmetric = np.allclose(A_symmetric, A_symmetric.T)  
    print(f"Is matrix symmetric? {is_symmetric}")  
  
    # Spectral decomposition  
    eigenvalues_sym, eigenvectors_sym, A_rec_sym =  
        ↪ spectral_decomposition(A_symmetric)  
  
    print(f"\nEigenvalues (should be real): {eigenvalues_sym}")  
    print(f"Eigenvalues are real? {np.all(np.isreal(eigenvalues_sym))}")
```

```

# Verify orthogonality of eigenvectors
print(f"\nEigenvectors:\n{eigenvectors_sym}")

print("\nOrthogonality Check (dot products):")
for i in range(3):
    for j in range(i+1, 3):
        dot_product = np.dot(eigenvectors_sym[:, i], eigenvectors_sym[:, j])
        print(f"v{i+1} · v{j+1} = {dot_product:.2e}")

# For symmetric matrices, eigenvectors should be orthogonal
# Check if QT * Q = I
orthogonality_check = eigenvectors_sym.T @ eigenvectors_sym
print(f"\nQT * Q (should be identity):\n{orthogonality_check}")
print(f"Orthogonality preserved? {np.allclose(orthogonality_check, np.eye(3))}")

# Compare with non-symmetric matrix
A_nonsymmetric = np.array([[1, 2, 3],
                             [0, 4, 5],
                             [0, 0, 6]])

print(f"\n--- Comparison with Non-Symmetric Matrix ---")
print(f"Non-symmetric Matrix:\n{A_nonsymmetric}")
eigenvalues_ns, eigenvectors_ns, _ = spectral_decomposition(A_nonsymmetric)
print(f"Eigenvalues: {eigenvalues_ns}")
print(f"Eigenvalues real? {np.all(np.isreal(eigenvalues_ns))}")

# Check condition numbers
cond_sym = np.linalg.cond(eigenvectors_sym)
cond_ns = np.linalg.cond(eigenvectors_ns)
print(f"\nCondition number of eigenvector matrix (symmetric): {cond_sym:.4f}")
print(f"Condition number of eigenvector matrix (non-symmetric): {cond_ns:.4f}")
print("Lower condition number indicates better numerical stability.")

```

```

=====
SYMMETRIC MATRICES
=====

```

Symmetric Matrix A:

```

[[6 2 1]
 [2 3 1]
 [1 1 2]]

```

Is matrix symmetric? True

Eigenvalues (should be real): [7.33949226 2.29511673 1.36539102]

Eigenvalues are real? True

Eigenvectors:

```
[[ 0.85772795  0.50752384  0.08198967]
 [ 0.45183049 -0.6681011  -0.59117691]
 [ 0.24525898 -0.54411439  0.80236373]]
```

Orthogonality Check (dot products):

$v_1 \cdot v_2 = 2.78e-16$

$v_1 \cdot v_3 = 8.33e-17$

$v_2 \cdot v_3 = 2.78e-16$

$Q^T * Q$ (should be identity):

```
[[1.00000000e+00 2.70093238e-16 9.33273463e-17]
 [2.70093238e-16 1.00000000e+00 2.66411610e-16]
 [9.33273463e-17 2.66411610e-16 1.00000000e+00]]
```

Orthogonality preserved? True

--- Comparison with Non-Symmetric Matrix ---

Non-symmetric Matrix:

```
[[1 2 3]
 [0 4 5]
 [0 0 6]]
```

Eigenvalues: [1. 4. 6.]

Eigenvalues real? True

Condition number of eigenvector matrix (symmetric): 1.0000

Condition number of eigenvector matrix (non-symmetric): 6.8066

Lower condition number indicates better numerical stability.